Module 8 : Signal Discovery and Alpha Generation

Learning Objectives

By the end of this module, you will:

1. Understand how to convert fair value estimates into trading signals
2. Master the `SignalGenerator` class for creating z-score normalized signals
3. Estimate mean reversion half-lives and their trading implications
4. Implement position sizing with the Kelly criterion
5. Apply risk constraints through the `RiskManager` class
6. Interpret forward curve divergence problems
7. Build a complete end-to-end trading system

The Critical Distinction

"Fair value is NOT a trading signal."

You can have:

- Accurate fair value estimates ✓
- Validated models ✓
- Clean features ✓

But without proper signal generation and risk management, you'll still **lose money** because:

1. You don't know **when** to trade
2. You don't know **how much** to trade
3. You don't know **when to exit**

In [1]:
```
# Standard imports
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime, timedelta
from sklearn.linear_model import LinearRegression
import warnings
warnings.filterwarnings('ignore')

# Set style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")

# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.precision', 4)
```

```
print("✓ Environment configured")
```

```
✓ Environment configured
```

In [2]:
```
# Import fair_value_toolkit
import sys
sys.path.insert(0, '../../src')

from fair_value_toolkit.signals import (
    SignalGenerator,
    PositionSizer,
    RiskManager,
    FairValueSignal,
    generate_signal_report
)
from fair_value_toolkit.models import (
    InventoryFairValueModel,
    SupplyDemandModel
)
from fair_value_toolkit.features import create_commodity_features

print("✓ fair_value_toolkit imported")
```

```
✓ fair_value_toolkit imported
```

Part 1: From Fair Value to Trading Signal

The Signal Generation Process

```
Fair Value Estimate → Mispricing → Z-Score → Thresholds → Trading
Signal
    ($65.50)          ($5.50)      (2.2σ)      (>1.5σ)        LONG
```

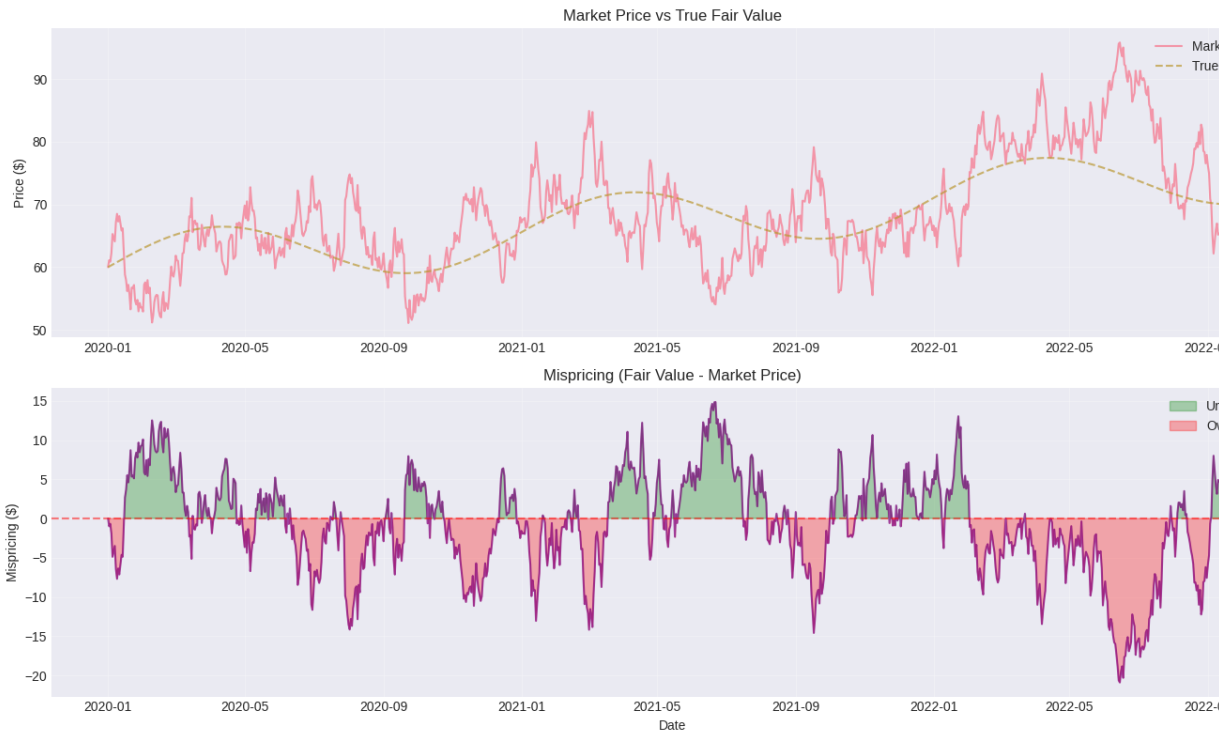
Key Components

1. **Mispricing**: `fair_value - market_price`
2. **Z-Score**: `(mispricing - μ) / σ` (standardized deviation)
3. **Entry Threshold**: Min z-score to open position (e.g., ± 1.5 σ)
4. **Exit Threshold**: Z-score to close position (e.g., ± 0.5 σ)
5. **Signal Strength**: Continuous value 0-1 based on z-score magnitude

In [3]:
```
# Create synthetic commodity data with mispricing opportunities
np.random.seed(42)
n = 1000
dates = pd.date_range('2020-01-01', periods=n, freq='D')

# True fair value (with trend)
true_fair_value = 60 + np.linspace(0, 15, n) + 5 * np.sin(2 * np.pi * np.

# Market price deviates from fair value
# Add mean-reverting mispricing
mispricing = np.zeros(n)
for i in range(1, n):
    # AR(1) with mean reversion
    mispricing[i] = 0.95 * mispricing[i-1] + np.random.normal(0, 2)
```

```
market_price = true_fair_value + mispricing

# Create DataFrame
df = pd.DataFrame({
    'market_price': market_price,
    'true_fair_value': true_fair_value,
    'inventory': 1000 + np.random.normal(0, 100, n).cumsum(),
    'production': 100 + np.random.normal(0, 5, n),
    'consumption': 98 + np.random.normal(0, 5, n),
}, index=dates)

print(f"Created synthetic market data: {len(df)} days")
print(f"Date range: {df.index[0].date()} to {df.index[-1].date()}")

# Visualize
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Prices
axes[0].plot(df.index, df['market_price'], label='Market Price', alpha=0.
axes[0].plot(df.index, df['true_fair_value'], label='True Fair Value', al
axes[0].set_ylabel('Price ($)')
axes[0].set_title('Market Price vs True Fair Value')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Mispricing
true_mispricing = df['true_fair_value'] - df['market_price']
axes[1].plot(df.index, true_mispricing, alpha=0.7, color='purple')
axes[1].axhline(0, color='red', linestyle='--', alpha=0.5)
axes[1].fill_between(df.index, 0, true_mispricing,
                     where=(true_mispricing > 0), alpha=0.3, color='gree
axes[1].fill_between(df.index, 0, true_mispricing,
                     where=(true_mispricing < 0), alpha=0.3, color='red'
axes[1].set_ylabel('Mispricing ($)')
axes[1].set_xlabel('Date')
axes[1].set_title('Mispricing (Fair Value - Market Price)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print(f"\nMispricing Statistics:")
print(f"  Mean: ${true_mispricing.mean():.2f}")
print(f"  Std: ${true_mispricing.std():.2f}")
print(f"  Max: ${true_mispricing.max():.2f}")
print(f"  Min: ${true_mispricing.min():.2f}")
```

```
Created synthetic market data: 1000 days
Date range: 2020-01-01 to 2022-09-26
```



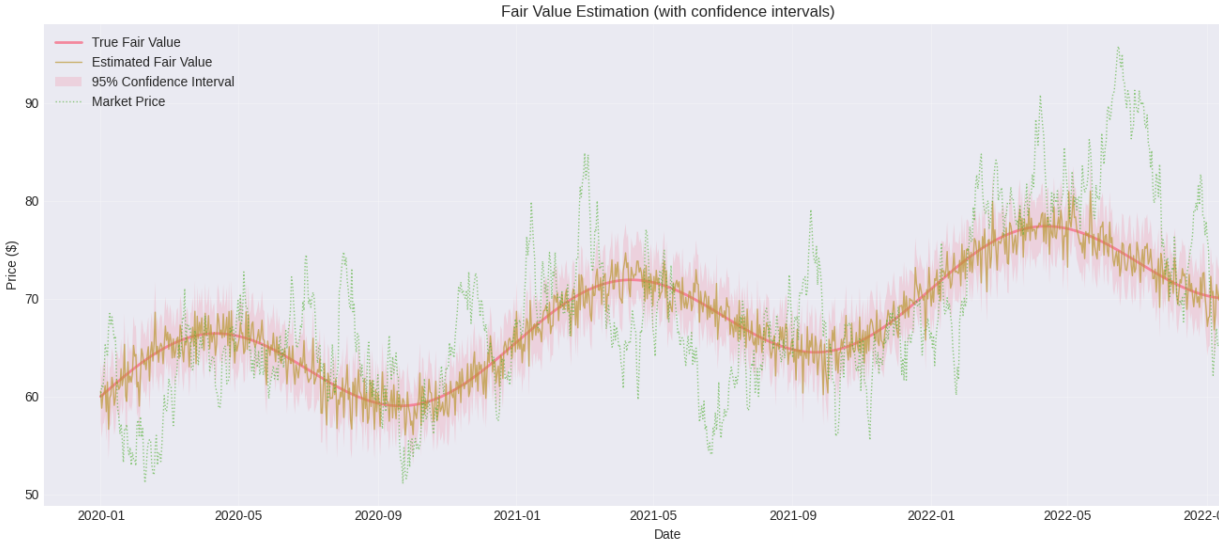

```
Mispricing Statistics:
  Mean: $-0.82
  Std: $6.32
  Max: $14.86
  Min: $-20.90
```

In [4]:

```python
# In reality, we must estimate fair value
# Add estimation error to simulate model predictions
estimation_error = np.random.normal(0, 1.5, n)
estimated_fair_value = df['true_fair_value'] + estimation_error

# Add confidence intervals (wider when less certain)
ci_width = 3.0  # ±$3 confidence interval
estimated_lower = estimated_fair_value - ci_width
estimated_upper = estimated_fair_value + ci_width

fair_value_df = pd.DataFrame({
    'fair_value': estimated_fair_value,
    'lower': estimated_lower,
    'upper': estimated_upper,
}, index=dates)

print("✓ Fair value estimates created (with realistic estimation error)")

# Visualize estimation quality
fig, ax = plt.subplots(figsize=(14, 6))

ax.plot(df.index, df['true_fair_value'], label='True Fair Value', linewid
ax.plot(df.index, fair_value_df['fair_value'], label='Estimated Fair Valu
ax.fill_between(df.index, fair_value_df['lower'], fair_value_df['upper'],
                alpha=0.2, label='95% Confidence Interval')
ax.plot(df.index, df['market_price'], label='Market Price', linewidth=1,

ax.set_ylabel('Price ($)')
ax.set_xlabel('Date')
ax.set_title('Fair Value Estimation (with confidence intervals)')
ax.legend()
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()

# Measure estimation quality
estimation_rmse = np.sqrt(np.mean((estimated_fair_value - df['true_fair_v
print(f"\nFair Value Estimation Quality:")
print(f"  RMSE: ${estimation_rmse:.2f}")
print(f"  Correlation with true value: {np.corrcoef(estimated_fair_value,
```

```
✓ Fair value estimates created (with realistic estimation error)
```




```
Fair Value Estimation Quality:
  RMSE: $1.49
  Correlation with true value: 0.962
```

Part 2 : Z-Score Normalization and Thresholds

Why Z-Scores?

Raw mispricing of $ 5 could mean:

- **Massive opportunity** if typical mispricing is $ 1
- **Noise** if typical mispricing is $ 1 0

Z-scores standardize: `z = (x - μ) / σ`

Threshold Selection

- **Entry: ± 1 . 5 σ to ± 2 σ** - Requires significant mispricing
- **Exit: ± 0 . 5 σ** - Close when mostly converged
- **Emergency exit: ± 3 σ** - Cut losses if diverging further

In [5]:
```
# Initialize SignalGenerator
signal_generator = SignalGenerator(
    entry_threshold=1.5,       # Enter when |z| > 1.5
    exit_threshold=0.5,        # Exit when |z| < 0.5
    lookback_window=252,       # 1 year for statistics
    confidence_weight=True,    # Weight by model confidence
    max_signal=1.0
)
```

```
print("✓ SignalGenerator initialized")
print(f"  Entry threshold: ±{signal_generator.entry_threshold}σ")
print(f"  Exit threshold: ±{signal_generator.exit_threshold}σ")
print(f"  Lookback window: {signal_generator.lookback_window} periods")
```

```
✓ SignalGenerator initialized
  Entry threshold: ±1.5σ
  Exit threshold: ±0.5σ
  Lookback window: 252 periods
```

In [6]:

```
# Generate signals
signals = signal_generator.generate(
    fair_values=fair_value_df,
    market_prices=df['market_price'],
    commodity='crude_oil'
)

print(f"✓ Signals generated: {len(signals)} observations")
print(f"\nSignal columns: {list(signals.columns)}")
print(f"\nSample signals:")
print(signals.tail(10))
```

```
✓ Signals generated: 1000 observations

Signal columns: ['fair_value', 'market_price', 'mispricing', 'mispricing_zsco
'signal_direction', 'signal_strength', 'confidence', 'commodity']

Sample signals:
            fair_value  market_price  mispricing  mispricing_zscore  \
2022-09-17     73.6064       68.4114      5.1950             1.4884
2022-09-18     66.8638       68.9031     -2.0393             0.4204
2022-09-19     71.0337       64.8712      6.1625             1.6161
2022-09-20     69.8342       64.6312      5.2030             1.4686
2022-09-21     70.8555       63.5349      7.3205             1.7700
2022-09-22     69.0196       61.8556      7.1640             1.7401
2022-09-23     69.9341       61.7027      8.2314             1.8857
2022-09-24     71.0780       65.7166      5.3613             1.4690
2022-09-25     74.6850       67.2178      7.4672             1.7650
2022-09-26     71.2339       66.2216      5.0123             1.4083

            signal_direction  signal_strength  confidence  commodity
2022-09-17                 0           0.4561      0.9194  crude_oil
2022-09-18                 0           0.1289      0.9199  crude_oil
2022-09-19                 1           0.4931      0.9153  crude_oil
2022-09-20                 0           0.4479      0.9151  crude_oil
2022-09-21                 1           0.5391      0.9137  crude_oil
2022-09-22                 1           0.5287      0.9116  crude_oil
2022-09-23                 1           0.5729      0.9114  crude_oil
2022-09-24                 0           0.4487      0.9163  crude_oil
2022-09-25                 1           0.5401      0.9181  crude_oil
2022-09-26                 0           0.4304      0.9169  crude_oil
```

In [7]:

```
# Visualize signals
fig, axes = plt.subplots(4, 1, figsize=(14, 12))

# 1. Price and fair value
axes[0].plot(signals.index, signals['market_price'], label='Market Price'
axes[0].plot(signals.index, signals['fair_value'], label='Fair Value', al
axes[0].set_ylabel('Price ($)')
axes[0].set_title('Market Price vs Fair Value Estimate')
axes[0].legend()
```

```
axes[0].grid(True, alpha=0.3)

# 2. Mispricing and z-score
ax2a = axes[1]
ax2b = ax2a.twinx()
ax2a.bar(signals.index, signals['mispricing'], alpha=0.3, label='Misprici
ax2b.plot(signals.index, signals['mispricing_zscore'], color='orange', la
ax2b.axhline(signal_generator.entry_threshold, color='red', linestyle='--
ax2b.axhline(-signal_generator.entry_threshold, color='green', linestyle=
ax2b.axhline(0, color='black', linestyle='-', alpha=0.3)
ax2a.set_ylabel('Mispricing ($)', color='gray')
ax2b.set_ylabel('Z-Score', color='orange')
ax2a.set_title('Mispricing and Z-Score Normalization')
ax2a.grid(True, alpha=0.3)

# 3. Signal direction
colors = ['green' if s == 1 else 'red' if s == -1 else 'gray'
          for s in signals['signal_direction']]
axes[2].bar(signals.index, signals['signal_direction'], color=colors, alp
axes[2].set_ylabel('Signal Direction')
axes[2].set_yticks([-1, 0, 1])
axes[2].set_yticklabels(['Short', 'Neutral', 'Long'])
axes[2].set_title('Trading Signal Direction')
axes[2].grid(True, alpha=0.3, axis='y')

# 4. Signal strength
axes[3].fill_between(signals.index, 0, signals['signal_strength'],
                     where=(signals['signal_direction'] == 1), alpha=0.5,
axes[3].fill_between(signals.index, 0, -signals['signal_strength'],
                     where=(signals['signal_direction'] == -1), alpha=0.5
axes[3].set_ylabel('Signal Strength')
axes[3].set_xlabel('Date')
axes[3].set_title('Signal Strength (0-1)')
axes[3].legend()
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Signal statistics
print("\nSignal Statistics:")
print(f"  Total signals: {len(signals)}")
print(f"  Long signals: {(signals['signal_direction'] == 1).sum()} ({(sig
print(f"  Short signals: {(signals['signal_direction'] == -1).sum()} ({(s
print(f"  Neutral periods: {(signals['signal_direction'] == 0).sum()} ({(
print(f"  Average signal strength (when active): {signals[signals['signal
```

```
Signal Statistics:
  Total signals: 1000
  Long signals: 63 (6.3%)
  Short signals: 113 (11.3%)
  Neutral periods: 824 (82.4%)
  Average signal strength (when active): 0.615
```

Part 3 : Mean Reversion Half-Life

Half-life = Time for mispricing to decay by 50%

If half-life = 5 days:

- Day 0 : Mispricing = $ 4
- Day 5 : Mispricing = $ 2
- Day 10 : Mispricing = $ 1

Trading Implications

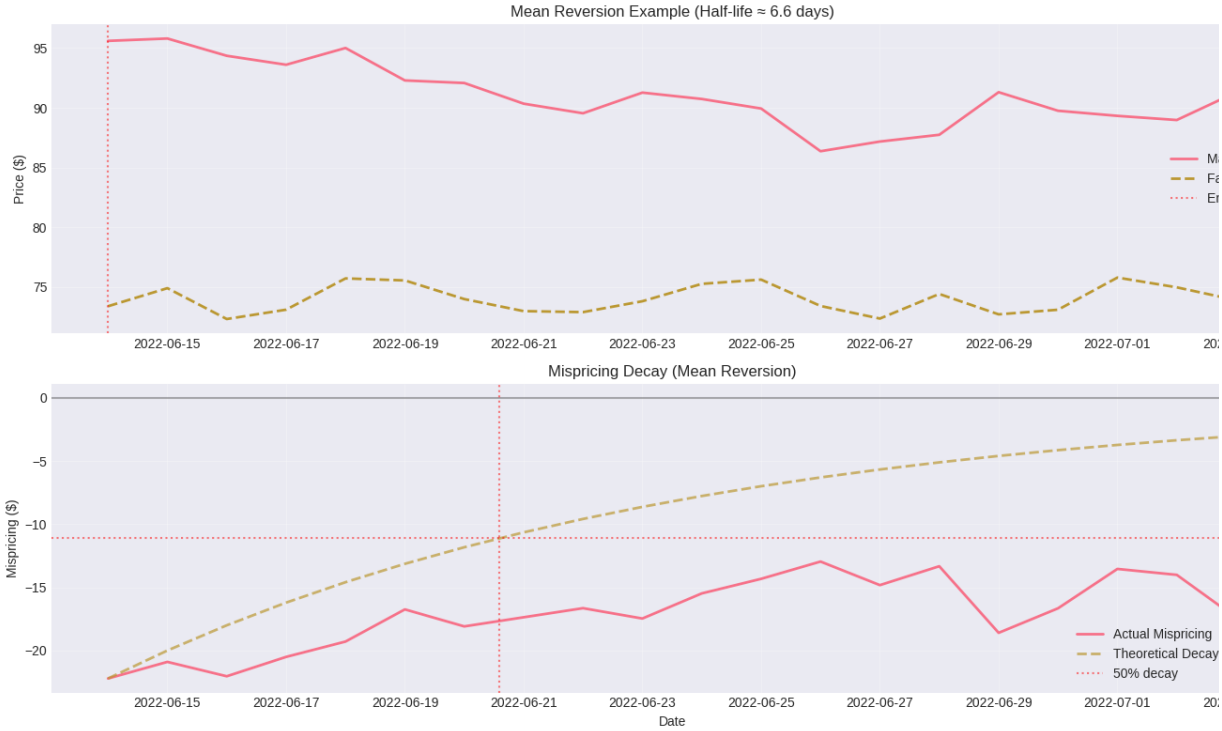
- **Short half-life** (1 - 5 days) → High-frequency trading, tight stops
- **Medium half-life** (5 - 20 days) → Swing trading, wider stops
- **Long half-life** (> 20 days) → Position trading, patience required
- **Infinite half-life** → No mean reversion! Don't trade

In [8]:
```
# Estimate half-life
half_life = signal_generator.estimate_half_life(signals['mispricing'])

print(f"\nEstimated Mean Reversion Half-Life:")
print(f"  {half_life:.1f} days")

if half_life < 5:
    print("  ⚡ Fast mean reversion - High-frequency trading")
    print("     Recommended holding period: 2-5 days")
elif half_life < 20:
    print("  🔄 Medium mean reversion - Swing trading")
    print(f"     Recommended holding period: {int(half_life)}-{int(half_l
elif half_life < 100:
    print("  📈 Slow mean reversion - Position trading")
    print(f"     Recommended holding period: {int(half_life)}-{int(half_l
else:
    print("  ⚠️  Very slow or no mean reversion!")
    print("     Consider momentum strategy instead")
```

```
Estimated Mean Reversion Half-Life:
  6.6 days
  🔄 Medium mean reversion - Swing trading
     Recommended holding period: 6-13 days
```

In [9]:
```
# Visualize mean reversion
# Take a sample period with large mispricing
large_mispricing_idx = signals['mispricing_zscore'].abs().idxmax()
sample_start = large_mispricing_idx
sample_end = sample_start + pd.Timedelta(days=int(half_life * 3))

sample_signals = signals.loc[sample_start:sample_end]

fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Price convergence
axes[0].plot(sample_signals.index, sample_signals['market_price'], label=
axes[0].plot(sample_signals.index, sample_signals['fair_value'], label='F
axes[0].axvline(sample_start, color='red', linestyle=':', alpha=0.5, labe
axes[0].set_ylabel('Price ($)')
axes[0].set_title(f'Mean Reversion Example (Half-life ≈ {half_life:.1f} d
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Mispricing decay
initial_mispricing = sample_signals.iloc[0]['mispricing']
theoretical_decay = initial_mispricing * np.exp(-np.log(2) / half_life *

axes[1].plot(sample_signals.index, sample_signals['mispricing'], label='A
axes[1].plot(sample_signals.index, theoretical_decay, label=f'Theoretical
             linewidth=2, linestyle='--', alpha=0.7)
axes[1].axhline(0, color='black', linestyle='-', alpha=0.3)
axes[1].axhline(initial_mispricing * 0.5, color='red', linestyle=':', alp
axes[1].axvline(sample_start + pd.Timedelta(days=half_life), color='red',
axes[1].set_ylabel('Mispricing ($)')
axes[1].set_xlabel('Date')
axes[1].set_title('Mispricing Decay (Mean Reversion)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```



Part 4 : Position Sizing with Kelly Criterion

The Question: How much capital should I allocate to each signal?

Position Sizing Methods

1. **Equal Weight**: All signals get same size (simple, inefficient)
2. **Signal Proportional**: Size ∝ signal strength (better, ignores risk)
3. **Kelly Criterion**: Optimal size = μ / σ² (mathematically optimal)
4. **Risk Parity**: Size ∝ 1 /volatility (focuses on risk balance)

Kelly Formula

```
Optimal Position Size = (Expected Return) / (Variance of Returns)
                      = μ / σ²
```

In practice, use **fractional Kelly** (e.g., 25 %) to be conservative.

In [10]:
```
# Initialize PositionSizer with different methods
position_sizers = {
    'Equal Weight': PositionSizer(
        method='equal',
        max_position=0.1  # Max 10% per position
    ),
    'Signal Proportional': PositionSizer(
        method='signal',
        max_position=0.1
    ),
    'Risk Parity': PositionSizer(
        method='risk_parity',
```

```
        max_position=0.1,
        target_volatility=0.15
    )
}

print("✓ Position sizers initialized")

# Calculate historical volatility
returns = df['market_price'].pct_change()
rolling_vol = returns.rolling(window=21).std() * np.sqrt(252)  # Annualiz
volatility = rolling_vol.reindex(signals.index).fillna(rolling_vol.mean()

print(f"\nVolatility Statistics:")
print(f"  Mean: {volatility.mean():.1%}")
print(f"  Min: {volatility.min():.1%}")
print(f"  Max: {volatility.max():.1%}")
```

```
✓ Position sizers initialized

Volatility Statistics:
  Mean: 46.5%
  Min: 24.7%
  Max: 73.6%
```

In [11]:

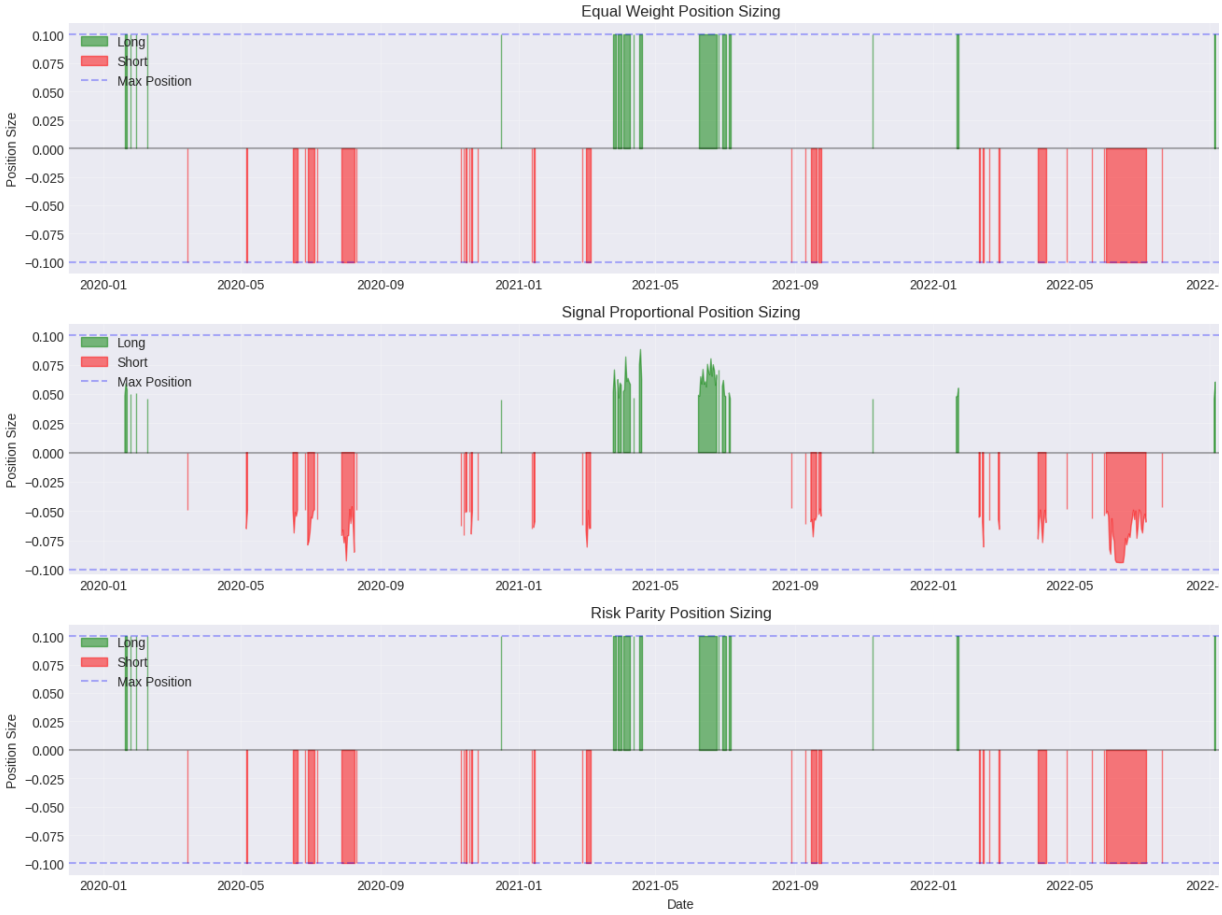
```
# Generate position sizes with each method
positions = {}

for name, sizer in position_sizers.items():
    if name == 'Risk Parity':
        positions[name] = sizer.size(signals, volatility=volatility)
    else:
        positions[name] = sizer.size(signals)

    print(f"\n{name}:")
    print(f"  Average position size: {positions[name].abs().mean():.3f}")
    print(f"  Max position size: {positions[name].abs().max():.3f}")
    print(f"  % of time invested: {(positions[name] != 0).sum() / len(pos

positions_df = pd.DataFrame(positions)
```

```
Equal Weight:
  Average position size: 0.018
  Max position size: 0.100
  % of time invested: 17.6%

Signal Proportional:
  Average position size: 0.011
  Max position size: 0.094
  % of time invested: 19.5%

Risk Parity:
  Average position size: 0.018
  Max position size: 0.100
  % of time invested: 19.5%
```

In [12]:

```
# Visualize position sizing methods
fig, axes = plt.subplots(3, 1, figsize=(14, 10))

for idx, (name, pos) in enumerate(positions.items()):
    axes[idx].fill_between(pos.index, 0, pos.values,
                           where=(pos.values > 0), alpha=0.5, color='gree
```

```
    axes[idx].fill_between(pos.index, 0, pos.values,
                           where=(pos.values < 0), alpha=0.5, color='red'
    axes[idx].axhline(0, color='black', linestyle='-', alpha=0.3)
    axes[idx].axhline(0.1, color='blue', linestyle='--', alpha=0.3, label
    axes[idx].axhline(-0.1, color='blue', linestyle='--', alpha=0.3)
    axes[idx].set_ylabel('Position Size')
    axes[idx].set_title(f'{name} Position Sizing')
    axes[idx].legend(loc='upper left')
    axes[idx].grid(True, alpha=0.3)

axes[-1].set_xlabel('Date')
plt.tight_layout()
plt.show()

# Compare sizing methods
print("\nComparison of Position Sizing Methods:")
print(positions_df.describe())
```




```
Comparison of Position Sizing Methods:
       Equal Weight  Signal Proportional  Risk Parity
count     1000.0000             981.0000     981.0000
mean        -0.0050              -0.0036      -0.0051
std          0.0417               0.0264       0.0421
min         -0.1000              -0.0941      -0.1000
25%          0.0000               0.0000       0.0000
50%          0.0000               0.0000       0.0000
75%          0.0000               0.0000       0.0000
max          0.1000               0.0880       0.1000
```

In [13]:

```
# Calculate returns for each sizing method
# Simplified: position * next_day_return
next_day_returns = df['market_price'].pct_change().shift(-1).reindex(sign
```

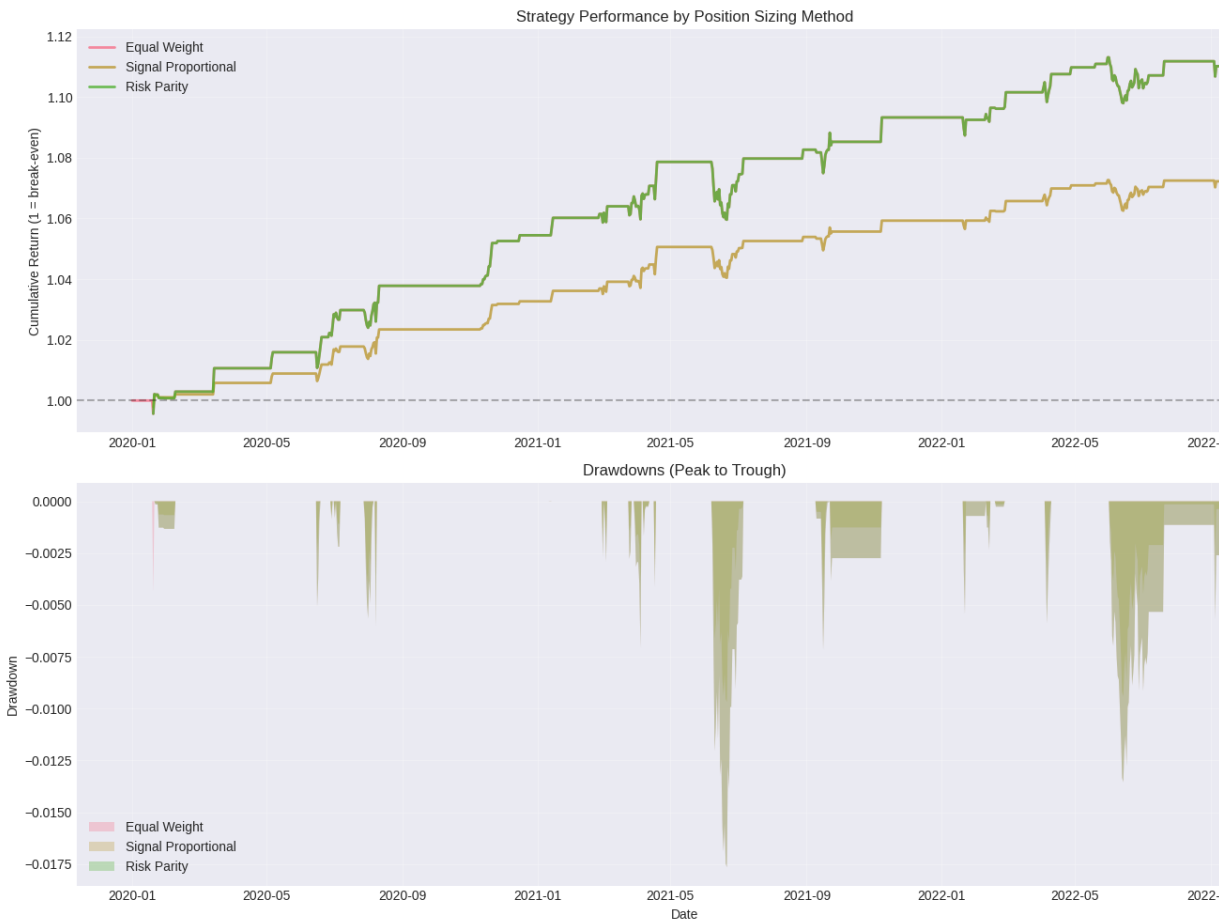
```
strategy_returns = {}
for name, pos in positions.items():
    strategy_returns[name] = pos * next_day_returns

returns_df = pd.DataFrame(strategy_returns)

# Calculate cumulative returns
cumulative_returns = (1 + returns_df).cumprod()

# Performance metrics
print("\nPerformance by Position Sizing Method:")
print("="*70)

for name in strategy_returns.keys():
    total_return = cumulative_returns[name].iloc[-1] - 1
    sharpe = returns_df[name].mean() / returns_df[name].std() * np.sqrt(2
    max_dd = (cumulative_returns[name] / cumulative_returns[name].cummax(

    print(f"\n{name}:")
    print(f"  Total Return: {total_return:.1%}")
    print(f"  Sharpe Ratio: {sharpe:.2f}")
    print(f"  Max Drawdown: {max_dd:.1%}")
    print(f"  Win Rate: {(returns_df[name] > 0).sum() / (returns_df[name]

print("="*70)
```

```
Performance by Position Sizing Method:
======================================================================

Equal Weight:
  Total Return: nan%
  Sharpe Ratio: 1.48
  Max Drawdown: -1.8%
  Win Rate: 58.2%

Signal Proportional:
  Total Return: nan%
  Sharpe Ratio: 1.63
  Max Drawdown: -1.0%
  Win Rate: 52.6%

Risk Parity:
  Total Return: nan%
  Sharpe Ratio: 1.49
  Max Drawdown: -1.8%
  Win Rate: 52.6%
======================================================================
```

In [14]:

```
# Visualize performance comparison
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Cumulative returns
for name in cumulative_returns.columns:
    axes[0].plot(cumulative_returns.index, cumulative_returns[name], labe
axes[0].axhline(1, color='black', linestyle='--', alpha=0.3)
axes[0].set_ylabel('Cumulative Return (1 = break-even)')
axes[0].set_title('Strategy Performance by Position Sizing Method')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
```

```
# Drawdowns
for name in cumulative_returns.columns:
    drawdown = cumulative_returns[name] / cumulative_returns[name].cummax
    axes[1].fill_between(drawdown.index, 0, drawdown.values, alpha=0.3, l
axes[1].set_ylabel('Drawdown')
axes[1].set_xlabel('Date')
axes[1].set_title('Drawdowns (Peak to Trough)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



Part 5 : Risk Management

The `RiskManager` applies constraints to prevent catastrophic losses:

1. **Position Limits**: Max single position (e.g., 10%)
2. **Total Exposure**: Max gross exposure (e.g., 100%)
3. **Correlation Constraints**: Avoid concentrated correlated bets
4. **Drawdown Controls**: Reduce size during losses
5. **VaR Limits**: Cap value at risk

In [15]:
```
# Initialize RiskManager
risk_manager = RiskManager(
    max_position_single=0.10,     # 10% max per position
    max_position_total=1.0,       # 100% gross exposure
    max_correlation=0.7,          # Max avg correlation
    drawdown_threshold=0.10       # Start reducing at 10% drawdown
```

```
)

print("✓ RiskManager initialized")
print(f"  Max single position: {risk_manager.max_position_single:.0%}")
print(f"  Max total exposure: {risk_manager.max_position_total:.0%}")
print(f"  Drawdown threshold: {risk_manager.drawdown_threshold:.0%}")
```

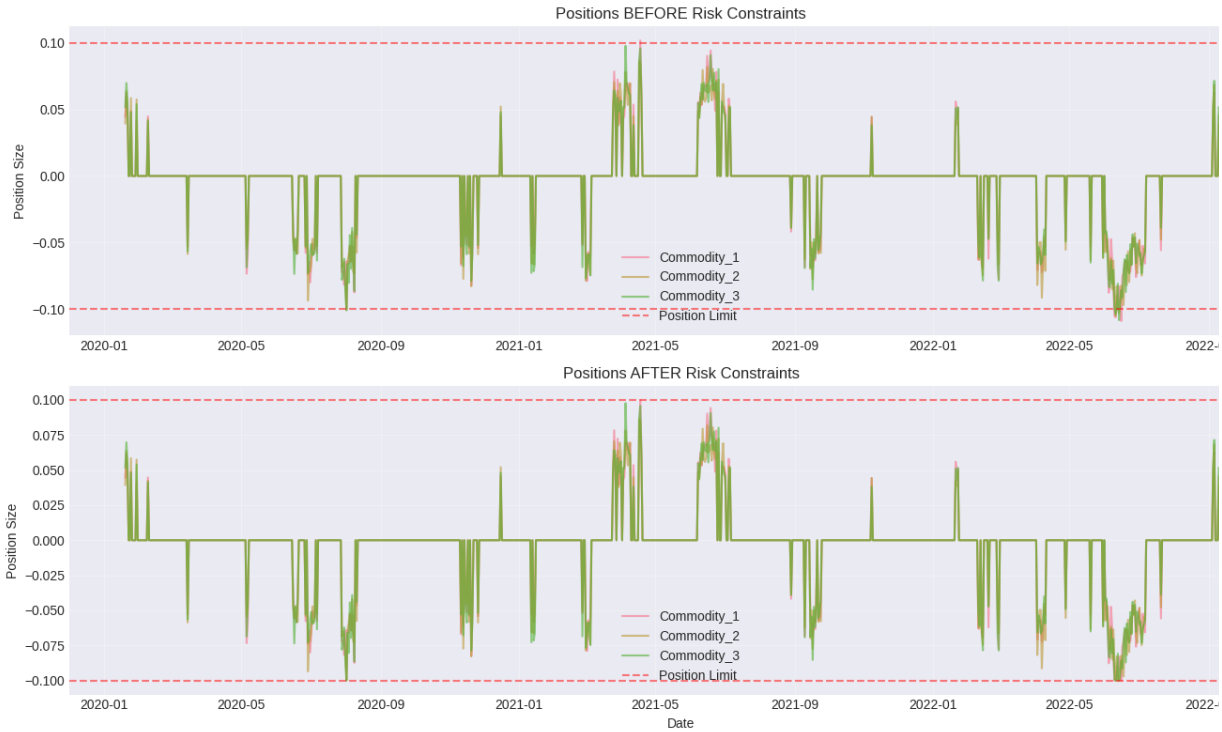
```
✓ RiskManager initialized
  Max single position: 10%
  Max total exposure: 100%
  Drawdown threshold: 10%
```

In [16]:

```
# Apply risk constraints
# For single commodity, most constraints don't apply
# Let's simulate a scenario with multiple positions

# Create a multi-position scenario (3 correlated commodities)
np.random.seed(42)
n_commodities = 3
multi_positions = pd.DataFrame({
    f'Commodity_{i+1}': positions['Signal Proportional'] * np.random.unif
    for i in range(n_commodities)
}, index=positions['Signal Proportional'].index)

print("\nUnconstrained Positions:")
print(f"  Total gross exposure: {multi_positions.abs().sum(axis=1).mean()
print(f"  Max single position: {multi_positions.abs().max().max():.3f}")

# Apply constraints
constrained_positions = risk_manager.apply_constraints(
    positions=multi_positions,
    current_drawdown=0.0  # No current drawdown
)

print("\nConstrained Positions:")
print(f"  Total gross exposure: {constrained_positions.abs().sum(axis=1).
print(f"  Max single position: {constrained_positions.abs().max().max():.

# Visualize constraints
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Before constraints
for col in multi_positions.columns:
    axes[0].plot(multi_positions.index, multi_positions[col], label=col,
axes[0].axhline(risk_manager.max_position_single, color='red', linestyle=
axes[0].axhline(-risk_manager.max_position_single, color='red', linestyle
axes[0].set_ylabel('Position Size')
axes[0].set_title('Positions BEFORE Risk Constraints')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# After constraints
for col in constrained_positions.columns:
    axes[1].plot(constrained_positions.index, constrained_positions[col],
axes[1].axhline(risk_manager.max_position_single, color='red', linestyle=
axes[1].axhline(-risk_manager.max_position_single, color='red', linestyle
axes[1].set_ylabel('Position Size')
axes[1].set_xlabel('Date')
axes[1].set_title('Positions AFTER Risk Constraints')
axes[1].legend()
```

```
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
Unconstrained Positions:
  Total gross exposure: 0.03
  Max single position: 0.109

Constrained Positions:
  Total gross exposure: 0.03
  Max single position: 0.100
```



In [17]:

```
# Calculate Value at Risk (VaR)
# Create sample returns for multiple commodities
commodity_returns = pd.DataFrame({
    f'Commodity_{i+1}': np.random.normal(0.0005, 0.02, len(multi_position
    for i in range(n_commodities)
}, index=multi_positions.index)

# Current positions (last row)
current_positions = constrained_positions.iloc[-1]

# Calculate VaR
var_95 = risk_manager.calculate_var(
    positions=current_positions,
    returns=commodity_returns,
    confidence=0.95,
    method='historical'
)

cvar_95 = risk_manager.calculate_cvar(
    positions=current_positions,
    returns=commodity_returns,
    confidence=0.95
)

print("\nRisk Metrics:")
```

```python
print("="*60)
print(f"Current Positions:")
for commodity, pos in current_positions.items():
    print(f"  {commodity}: {pos:.3f}")
print(f"\nValue at Risk (95% confidence):")
print(f"  1-day VaR: {var_95:.2%} of portfolio")
print(f"  Expected loss in worst 5% of cases")
print(f"\nConditional VaR (Expected Shortfall):")
print(f"  CVaR: {cvar_95:.2%} of portfolio")
print(f"  Average loss beyond VaR threshold")
print("="*60)
```

```
Risk Metrics:
============================================================
Current Positions:
  Commodity_1: 0.000
  Commodity_2: 0.000
  Commodity_3: 0.000

Value at Risk (95% confidence):
  1-day VaR: -0.00% of portfolio
  Expected loss in worst 5% of cases

Conditional VaR (Expected Shortfall):
  CVaR: -0.00% of portfolio
  Average loss beyond VaR threshold
============================================================
```

Part 6 : The Forward Curve Divergence Problem

The Dilemma

Scenario: Your fundamental model says crude oil is **undervalued** at $ 60 (fair value $ 70 futures curve is in **steep contango** (backwardation premium = -$ 5 /barrel).

```
Fundamentals say:  BUY  (price too low)
Forward curve says: SELL (contango = storage glut)
```

Interpreting Divergence

1. **Fundamental model is right, market is wrong**
   - Opportunity: Mean reversion trade
   - Risk: Market can stay irrational longer than you can stay solvent
2. **Market is right, fundamental model is wrong**
   - Warning: Missing information (hidden supply, demand shift)
   - Risk: Model breakdown, losses
3. **Both are right, timing mismatch**
   - Insight: Fundamental value = long-term, curve = short-term
   - Strategy: Adjust holding period or hedge

Signal from Divergence

The divergence itself can be a signal:

- **Large divergence** → Higher uncertainty → Reduce position size
- **Convergence** → Confirmation → Increase confidence
- **Persistent divergence** → Regime change → Re-estimate model

In [18]:
```
# Simulate forward curve data
# Create contango/backwardation signal
np.random.seed(42)

# Spot price
spot_price = df['market_price'].copy()

# Forward prices (1m, 3m, 6m, 12m)
forward_1m = spot_price + np.random.normal(0.5, 1.0, len(spot_price))  #
forward_3m = spot_price + np.random.normal(1.5, 1.5, len(spot_price))
forward_6m = spot_price + np.random.normal(2.0, 2.0, len(spot_price))
forward_12m = spot_price + np.random.normal(2.5, 2.5, len(spot_price))

# Calculate forward curve slope
curve_slope = (forward_12m - spot_price) / 12  # $/month

forward_df = pd.DataFrame({
    'spot': spot_price,
    'forward_1m': forward_1m,
    'forward_3m': forward_3m,
    'forward_6m': forward_6m,
    'forward_12m': forward_12m,
    'curve_slope': curve_slope,
}, index=df.index)

print("✓ Forward curve data created")
print(f"\nForward Curve Statistics:")
print(f"  Average curve slope: ${curve_slope.mean():.2f}/month")
print(f"  Contango periods: {(curve_slope > 0).sum()} ({(curve_slope > 0)
print(f"  Backwardation periods: {(curve_slope < 0).sum()} ({(curve_slope
```

```
✓ Forward curve data created

Forward Curve Statistics:
  Average curve slope: $0.20/month
  Contango periods: 826 (82.6%)
  Backwardation periods: 174 (17.4%)
```

In [19]:
```
# Visualize forward curve
fig, axes = plt.subplots(2, 1, figsize=(14, 10))

# Forward curve snapshots (sample dates)
sample_dates = [forward_df.index[i] for i in [100, 300, 500, 700]]
maturities = [0, 1, 3, 6, 12]  # months

for date in sample_dates:
    prices = [forward_df.loc[date, 'spot'],
              forward_df.loc[date, 'forward_1m'],
              forward_df.loc[date, 'forward_3m'],
```

```
                forward_df.loc[date, 'forward_6m'],
                forward_df.loc[date, 'forward_12m']]
    axes[0].plot(maturities, prices, 'o-', label=date.strftime('%Y-%m-%d'

axes[0].set_xlabel('Maturity (months)')
axes[0].set_ylabel('Price ($)')
axes[0].set_title('Forward Curve Snapshots')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Curve slope over time
axes[1].fill_between(forward_df.index, 0, forward_df['curve_slope'],
                     where=(forward_df['curve_slope'] > 0), alpha=0.5, co
axes[1].fill_between(forward_df.index, 0, forward_df['curve_slope'],
                     where=(forward_df['curve_slope'] < 0), alpha=0.5, co
axes[1].axhline(0, color='black', linestyle='-', alpha=0.5)
axes[1].set_ylabel('Curve Slope ($/month)')
axes[1].set_xlabel('Date')
axes[1].set_title('Forward Curve Slope Over Time')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

In [20]:
```
# Analyze divergence between fundamental signal and forward curve
# Combine signals and forward curve
divergence_df = signals.join(forward_df[['curve_slope']])

# Fundamental signal direction
divergence_df['fundamental_signal'] = divergence_df['signal_direction']
```

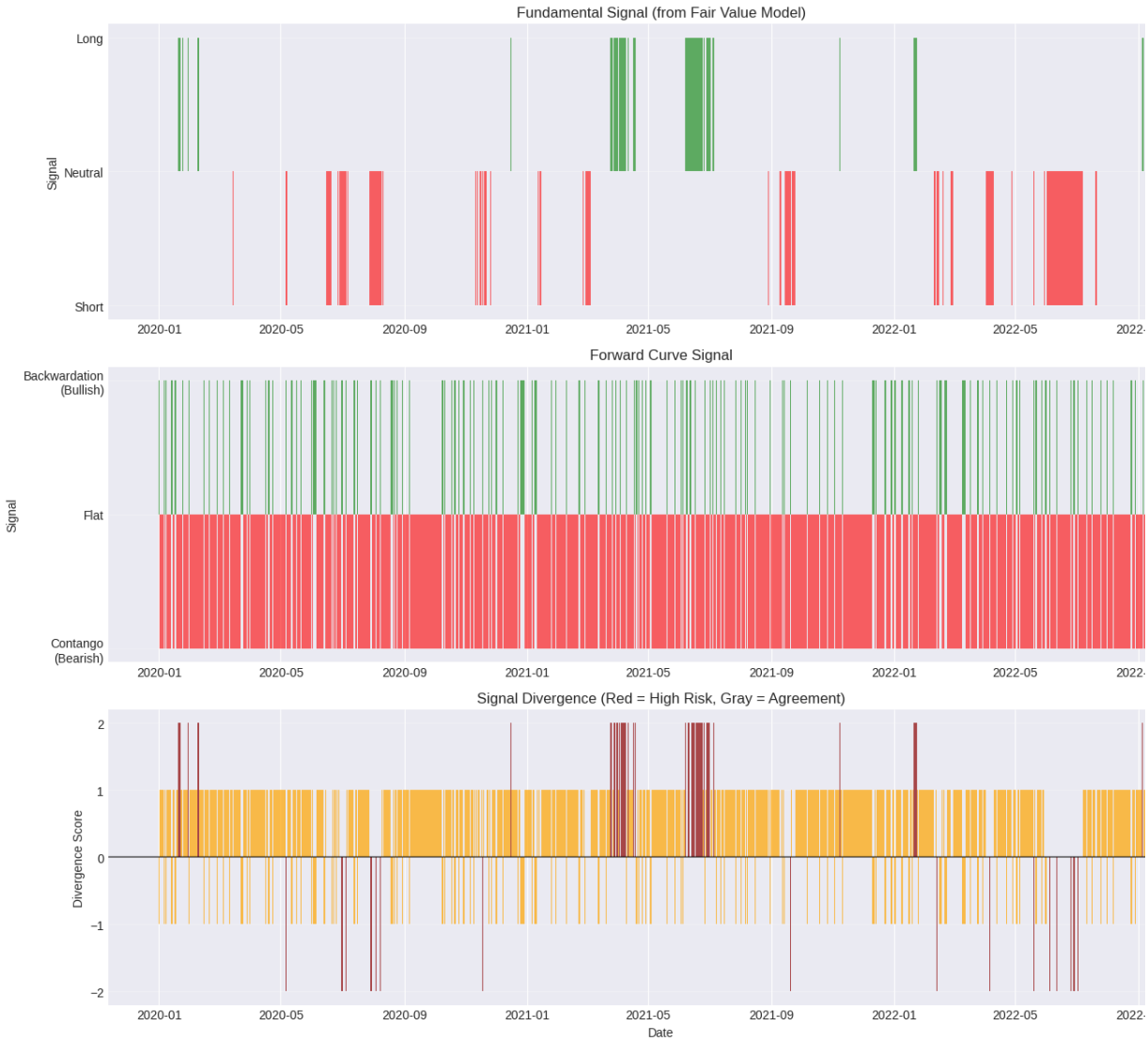
```
# Forward curve signal (contango = bearish, backwardation = bullish)
divergence_df['curve_signal'] = np.where(divergence_df['curve_slope'] < 0
                                         np.where(divergence_df['curve_sl

# Divergence score (-2 to +2)
divergence_df['divergence'] = divergence_df['fundamental_signal'] - diver

print("\nDivergence Analysis:")
print("="*60)
print(f"\nDivergence Distribution:")
print(divergence_df['divergence'].value_counts().sort_index())

print(f"\nInterpretation:")
print(f"  +2: Strong bullish fundamental + bearish curve (HIGH RISK)")
print(f"  +1: Moderate divergence")
print(f"   0: Agreement between fundamental and curve (HIGH CONFIDENCE)")
print(f"  -1: Moderate divergence")
print(f"  -2: Strong bearish fundamental + bullish curve (HIGH RISK)")
print("="*60)
```

```
Divergence Analysis:
============================================================

Divergence Distribution:
divergence
-2     16
-1    141
 0    114
 1    683
 2     46
Name: count, dtype: int64

Interpretation:
  +2: Strong bullish fundamental + bearish curve (HIGH RISK)
  +1: Moderate divergence
   0: Agreement between fundamental and curve (HIGH CONFIDENCE)
  -1: Moderate divergence
  -2: Strong bearish fundamental + bullish curve (HIGH RISK)
============================================================
```

In [21]:

```
# Visualize fundamental-curve divergence
fig, axes = plt.subplots(3, 1, figsize=(14, 12))

# Fundamental signal
axes[0].bar(divergence_df.index, divergence_df['fundamental_signal'],
           color=['green' if x > 0 else 'red' if x < 0 else 'gray' for x
           alpha=0.6, width=1)
axes[0].set_ylabel('Signal')
axes[0].set_yticks([-1, 0, 1])
axes[0].set_yticklabels(['Short', 'Neutral', 'Long'])
axes[0].set_title('Fundamental Signal (from Fair Value Model)')
axes[0].grid(True, alpha=0.3, axis='y')

# Forward curve signal
axes[1].bar(divergence_df.index, divergence_df['curve_signal'],
           color=['green' if x > 0 else 'red' if x < 0 else 'gray' for x
           alpha=0.6, width=1)
axes[1].set_ylabel('Signal')
axes[1].set_yticks([-1, 0, 1])
axes[1].set_yticklabels(['Contango\n(Bearish)', 'Flat', 'Backwardation\n(
```

```
axes[1].set_title('Forward Curve Signal')
axes[1].grid(True, alpha=0.3, axis='y')

# Divergence
divergence_colors = ['darkred' if abs(x) == 2 else 'orange' if abs(x) ==
                     for x in divergence_df['divergence']]
axes[2].bar(divergence_df.index, divergence_df['divergence'],
           color=divergence_colors, alpha=0.7, width=1)
axes[2].axhline(0, color='black', linestyle='-', alpha=0.5)
axes[2].set_ylabel('Divergence Score')
axes[2].set_xlabel('Date')
axes[2].set_yticks([-2, -1, 0, 1, 2])
axes[2].set_title('Signal Divergence (Red = High Risk, Gray = Agreement)'
axes[2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()
```



In [22]:

```
# Adjust positions based on divergence
# High divergence → reduce position size (higher uncertainty)

# Confidence adjustment factor
confidence_adjustment = 1.0 - (divergence_df['divergence'].abs() / 2) * 0
# |div| = 0 → factor = 1.0 (full size)
# |div| = 1 → factor = 0.75 (75% size)
# |div| = 2 → factor = 0.5 (50% size)
```

```python
# Adjusted positions
base_positions = positions['Signal Proportional'].reindex(divergence_df.i
adjusted_positions = base_positions * confidence_adjustment

print("\nPosition Adjustment Based on Divergence:")
print(f"  Original avg position: {base_positions.abs().mean():.3f}")
print(f"  Adjusted avg position: {adjusted_positions.abs().mean():.3f}")
print(f"  Average adjustment factor: {confidence_adjustment.mean():.2f}")

# Visualize adjustment
fig, axes = plt.subplots(2, 1, figsize=(14, 8))

# Confidence adjustment
axes[0].plot(confidence_adjustment.index, confidence_adjustment, alpha=0.
axes[0].axhline(1.0, color='green', linestyle='--', alpha=0.5, label='Ful
axes[0].axhline(0.5, color='red', linestyle='--', alpha=0.5, label='50% s
axes[0].set_ylabel('Adjustment Factor')
axes[0].set_title('Position Size Adjustment Based on Signal Divergence')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Original vs adjusted positions
axes[1].fill_between(base_positions.index, 0, base_positions,
                     alpha=0.3, label='Original Position', color='blue')
axes[1].fill_between(adjusted_positions.index, 0, adjusted_positions,
                     alpha=0.7, label='Adjusted Position', color='green')
axes[1].set_ylabel('Position Size')
axes[1].set_xlabel('Date')
axes[1].set_title('Position Sizing: Original vs Divergence-Adjusted')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
Position Adjustment Based on Divergence:
  Original avg position: 0.011
  Adjusted avg position: 0.009
  Average adjustment factor: 0.76
```

Part 7 : Complete Trading System

Let's put it all together into an end-to-end trading system:

```
Market Data → Fair Value Model → Signals → Position Sizing → Risk
Management → Orders
```

In [23]:

```
# Complete trading system
class CommodityTradingSystem:
    """End-to-end commodity trading system."""

    def __init__(self, signal_generator, position_sizer, risk_manager):
        self.signal_generator = signal_generator
        self.position_sizer = position_sizer
        self.risk_manager = risk_manager
        self.equity_curve = None

    def generate_orders(self, fair_values, market_prices, volatility=None
        """Generate trading orders from fair values."""

        # Step 1: Generate signals
        signals = self.signal_generator.generate(fair_values, market_pric

        # Step 2: Calculate base position sizes
        positions = self.position_sizer.size(signals, volatility=volatili

        # Step 3: Adjust for forward curve divergence (if available)
        if forward_curve is not None:
            # Calculate divergence adjustment
            curve_signal = np.where(forward_curve < 0, 1, np.where(forwar
            divergence = signals['signal_direction'] - curve_signal
            confidence_adj = 1.0 - (np.abs(divergence) / 2) * 0.5
            positions = positions * confidence_adj

        # Step 4: Apply risk constraints
        positions_df = pd.DataFrame({'commodity': positions})
```

```
        current_dd = self._calculate_current_drawdown()
        constrained_positions = self.risk_manager.apply_constraints(
            positions_df,
            current_drawdown=current_dd
        )

        return constrained_positions['commodity']

    def _calculate_current_drawdown(self):
        """Calculate current drawdown from equity curve."""
        if self.equity_curve is None or len(self.equity_curve) == 0:
            return 0.0
        peak = self.equity_curve.expanding().max().iloc[-1]
        current = self.equity_curve.iloc[-1]
        return (peak - current) / peak if peak > 0 else 0.0

    def backtest(self, fair_values, market_prices, returns, volatility=No
        """Run backtest of complete system."""

        # Generate orders
        positions = self.generate_orders(fair_values, market_prices, vola

        # Calculate returns
        strategy_returns = positions * returns

        # Build equity curve
        self.equity_curve = (1 + strategy_returns).cumprod()

        # Calculate metrics
        total_return = self.equity_curve.iloc[-1] - 1
        sharpe = strategy_returns.mean() / strategy_returns.std() * np.sq
        max_dd = (self.equity_curve / self.equity_curve.cummax() - 1).min
        win_rate = (strategy_returns > 0).sum() / (strategy_returns != 0)

        return {
            'total_return': total_return,
            'sharpe_ratio': sharpe,
            'max_drawdown': max_dd,
            'win_rate': win_rate,
            'positions': positions,
            'returns': strategy_returns,
            'equity_curve': self.equity_curve
        }

print("✓ CommodityTradingSystem class defined")
```

```
✓ CommodityTradingSystem class defined
```

In [24]:

```
# Initialize complete system
trading_system = CommodityTradingSystem(
    signal_generator=SignalGenerator(entry_threshold=1.5, exit_threshold=
    position_sizer=PositionSizer(method='risk_parity', max_position=0.1,
    risk_manager=RiskManager(max_position_single=0.10, drawdown_threshold
)

print("✓ Complete trading system initialized")

# Run backtest
results = trading_system.backtest(
    fair_values=fair_value_df,
```

```
    market_prices=df['market_price'],
    returns=next_day_returns,
    volatility=volatility,
    forward_curve=forward_df['curve_slope']
)

print("\n" + "="*70)
print("COMPLETE TRADING SYSTEM BACKTEST RESULTS")
print("="*70)
print(f"\nTotal Return: {results['total_return']:.1%}")
print(f"Sharpe Ratio: {results['sharpe_ratio']:.2f}")
print(f"Max Drawdown: {results['max_drawdown']:.1%}")
print(f"Win Rate: {results['win_rate']:.1%}")
print(f"\nNumber of trades: {(results['positions'].diff() != 0).sum()}")
print(f"Average position size: {results['positions'].abs().mean():.3f}")
print(f"Time in market: {(results['positions'] != 0).sum() / len(results[
print("="*70)
```

```
✓ Complete trading system initialized

======================================================================
COMPLETE TRADING SYSTEM BACKTEST RESULTS
======================================================================

Total Return: nan%
Sharpe Ratio: 1.57
Max Drawdown: -1.3%
Win Rate: 52.6%

Number of trades: 170
Average position size: 0.015
Time in market: 19.5%
======================================================================
```

In [25]:

```
# Visualize complete system performance
fig, axes = plt.subplots(3, 1, figsize=(14, 12))

# Equity curve
axes[0].plot(results['equity_curve'].index, results['equity_curve'], line
axes[0].axhline(1, color='black', linestyle='--', alpha=0.3, label='Break
axes[0].set_ylabel('Equity (1 = starting capital)')
axes[0].set_title('Equity Curve')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Drawdown
drawdown = results['equity_curve'] / results['equity_curve'].cummax() - 1
axes[1].fill_between(drawdown.index, 0, drawdown, alpha=0.5, color='red')
axes[1].axhline(trading_system.risk_manager.drawdown_threshold * -1,
               color='orange', linestyle='--', alpha=0.5, label='Drawdown
axes[1].set_ylabel('Drawdown')
axes[1].set_title('Drawdown from Peak')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Positions
axes[2].fill_between(results['positions'].index, 0, results['positions'],
                     where=(results['positions'] > 0), alpha=0.5, color='
axes[2].fill_between(results['positions'].index, 0, results['positions'],
                     where=(results['positions'] < 0), alpha=0.5, color='
```

```
axes[2].axhline(0, color='black', linestyle='-', alpha=0.3)
axes[2].set_ylabel('Position Size')
axes[2].set_xlabel('Date')
axes[2].set_title('Position Sizing Over Time')
axes[2].legend()
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

In [26]:

```
# Generate final signal report
signal_report = generate_signal_report(
    signals=signals,
    positions=results['positions'],
    returns=results['returns']
)

print("\nFinal Signal Report:")
print("="*70)
for key, value in signal_report.items():
    if isinstance(value, float):
        print(f"{key:30s}: {value:.4f}")
    else:
        print(f"{key:30s}: {value}")
print("="*70)
```

```
Final Signal Report:
======================================================================
total_signals                 : 1000
long_signals                  : 63
short_signals                 : 113
neutral_periods               : 824
avg_signal_strength           : 0.2692
avg_confidence                : 0.9186
avg_mispricing_pct            : -0.4008
avg_position                  : 0.0148
max_position                  : 0.1000
turnover                      : 0.0112
total_return                  : 0.0983
avg_return                    : 0.0001
sharpe                        : 1.5740
win_rate                      : 0.1030
======================================================================
```

Exercises

1. **Dynamic thresholds**: Implement adaptive entry/exit thresholds based on recent volatility
2. **Multi-timeframe signals**: Combine signals from different horizons (daily, weekly, monthly)
3. **Options hedging**: Use options Greeks to hedge tail risk in your positions
4. **Regime detection**: Build a regime detector and adjust parameters per regime
5. **Transaction costs**: Implement realistic transaction costs and optimize trade frequency

Key Takeaways

1. **Fair value ≠ trading signal** - You need threshold-based signal generation
2. **Z-score normalization is essential** - Standardizes mispricing across volatility regimes
3. **Mean reversion half-life determines holding period** - Fast reversion = short holds, slow = holds
4. **Kelly criterion provides optimal position sizing** - Use fractional Kelly to be conservative
5. **Risk management prevents catastrophic losses** - Position limits, exposure caps, drawdo controls
6. **Forward curve divergence signals uncertainty** - Reduce size when fundamentals and cu disagree
7. **Integrate all components into a trading system** - Signal → Size → Risk → Orders
8. **Always backtest with realistic assumptions** - Transaction costs, slippage, data availabilit

Course Conclusion

You now have a complete framework for:

- Building point-in-time datasets (Module 1 - 2)
- Estimating fair values (Module 3 - 5)
- Engineering features (Module 6)
- Validating models (Module 7)
- Generating trading signals (Module 8)

Next Steps:

1. Apply this framework to real commodity data
2. Test multiple commodities and build a portfolio
3. Deploy with proper monitoring and risk controls
4. Continuously validate and retrain models

Remember: The market is adversarial and adaptive. Your edge degrades over time. Stay discipl skeptical, and always questioning your assumptions.

Good luck!